

Infrastructure Scaling & Migration Report

Target Scale: 200 Restaurants, 5,000 Tables

1. Executive Summary

The current platform infrastructure runs entirely on Free Tier services (Vercel, Render, MongoDB Atlas). While cost-effective for initial development, this stack is mathematically guaranteed to fail under the load of 200 commercial restaurants. Peak dinner rushes will trigger hard rate limits, causing severe outages, dropped orders, and locked databases.

This report outlines the specific free-tier crash points and presents four progressive migration strategies—ranging from an ultra-low-cost dedicated server to a fully managed, enterprise-grade cloud architecture.

2. Current Architecture Crash Points

A. MongoDB Atlas (M0 Free Tier)

- Limitation: Hard-capped at 100 Operations/Sec and 512 MB Storage.
- Impact (Concurrency): At roughly 30-50 simultaneous active users, the database will throttle, resulting in massive queue delays and "504 Gateway Timeouts".
- Impact (Storage): At 20,000 orders/day, the 512 MB limit will fill within 30 days, forcing the database into a frozen Read-Only state.

B. Render Backend API (Free Tier)

- Limitation: 0.1 shared vCPU and 500 uptime hours per month.
- Impact (Performance): 0.1 vCPU cannot process more than 10-20 requests per second. Dinner rushes will max out the CPU, causing "502 Bad Gateway" errors.
- Impact (Uptime): 500 hours is only 20.8 days. If kept awake by traffic, the API will permanently shut down on the 21st day of every month.

C. Vercel Frontend (Hobby Tier)

- Limitation: 100 GB Bandwidth per month.
- Impact: Operating 5,000 tables will generate ~300,000 monthly visits. This consumes an estimated 600 GB of bandwidth. Vercel will hard-block the frontend upon exceeding the 100 GB limit.

3. Migration Strategies & Pricing Tiers

To support the target scale reliably, we present four distinct paths ranging from fully managed (hands-off) to self-hosted (hands-on).

Option A: The Enterprise Standard (Fully Managed PaaS)

- Frontend: Vercel Pro (\$20/mo)
- Backend: Render Standard (\$25/mo) - Dedicated 1 vCPU, 2GB RAM.
- Database: MongoDB Atlas M10 (\$60/mo) - 10GB Storage, 3,000 connections.
- Total Estimated Cost: \$105.00 / month

Pros: Zero server maintenance, automated disaster recovery, maximum reliability.

Cons: Highest monthly cost.

Option B: The "Smart Hybrid" (Mid-Tier Managed)

- Frontend: Cloudflare Pages (\$0/mo)
- Backend: Render Standard (\$25/mo) - Dedicated 1 vCPU, 2GB RAM.
- Database: DigitalOcean Managed MongoDB (\$15/mo) - 10GB Storage.
- Total Estimated Cost: \$40.00 / month

Pros: Fully managed (zero maintenance) like Option A, but saves \$65/mo by utilizing a cheaper database provider and free CDN.

Cons: DigitalOcean Mongo lacks some of the advanced analytics of Atlas.

Option C: The "Pay-As-You-Grow" (Serverless Starter)

- Frontend: Cloudflare Pages (\$0/mo)
- Backend: Render Starter (\$7/mo) - 0.5 vCPU, 512MB RAM, always on.
- Database: MongoDB Atlas Serverless (~\$5 to \$10/mo) - Pay exactly per read/write.
- Total Estimated Cost: ~\$12.00 - \$17.00 / month

Pros: The absolute lowest entry cost for a fully managed setup. Scales infinitely.

Cons: Backend API only gets 0.5 vCPU which may bottleneck during massive traffic spikes. Database costs are variable and unpredictable.

Option D: The Dedicated VPS (Self-Hosted Monolith)

- Frontend, Backend, & DB: Single DigitalOcean Droplet (\$24/mo) - 2 vCPUs, 4GB RAM, 80GB NVMe storage.
- Total Estimated Cost: \$24.00 / month

Pros: Massive compute and storage (80GB) for a flat, predictable fee.

Cons: Requires a DevOps engineer to manually manage security, SSL, and database backups.

4. Long-Term Data Archiving Strategy

As the platform grows to millions of orders per year, retaining all historical data in the primary operational database becomes expensive and degrades query performance. We recommend implementing an Archiving Strategy after 12 months of operation:

The "Cold Storage" Archiving Model

1. Background Sweeps: A scheduled background job runs monthly, identifying orders older than 6 months.
2. Export to Cold Storage: These orders are serialized into JSON/CSV files and moved to cheap object storage (e.g., AWS S3), costing fractions of a penny.
3. Database Purge: The old orders are deleted from MongoDB, keeping the live database ultra-lean and incredibly fast.
4. Historical Retrieval: If a restaurant owner requires a report from 2 years ago, they request it via the UI. The system compiles the archived files into a spreadsheet and emails it to them automatically.

5. Conclusion & Recommendation

Attempting to launch 200 restaurants on the current Free Tier architecture will result in immediate, catastrophic platform failure during the first peak traffic event.

Recommendation: Option B (The "Smart Hybrid" at \$40/mo) strikes the perfect balance for a growing startup. It completely eliminates server maintenance and database management while remaining highly affordable. Option C is a great temporary stepping-stone, but Option B provides the dedicated CPU necessary to guarantee API performance during peak restaurant dinner rushes.